

Week 6: Memory & Self-Healing (EEPROM & The Watchdog)

IoT Bhai: RoboStart | Master Embedded C++

Mentors: Md. Kamruzzaman Tipu &
Md. Shohanuzzaman

Training Objectives



Understand the Memory Regions of the ATmega328P (Flash, SRAM, EEPROM)



Perform Read, Write, Update, Put, and Get Operations



Distinguish Between *write()* and *update()* for EEPROM Lifespan



Implement a **Boot Counter** that Survives Power Cycles



Learn **Watchdog Timer** for System Recovery



Configure and Reset the **Watchdog Timer**



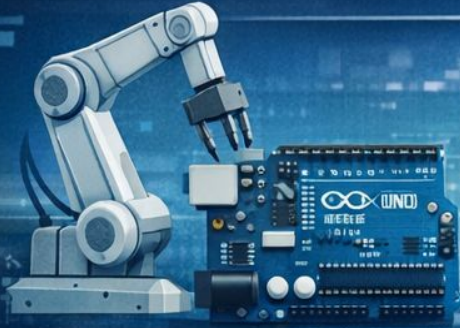
Build Crash-Resilient, *Self-Healing Systems*



Combine EEPROM and Watchdog for *Persistent Solutions*



ATmega328P Memory Regions



Memory Type	Size	Access Speed	Endurance	Purpose
Flash	32 KB	~ 100 ns (read)	10,000 write cycles	Stores the compiled code.
SRAM	2 KB	~ 10 ns (read/write)	Unlimited	Runtime variables & stack.
EEPROM	1 KB	~ 3.3 ms (write)	100,000 write cycles	User data, survives power-off.

EEPROM – Anatomy and Access Overview

EEPROM Size: 1024 Bytes (Addresses 0~1023)

Address	Size	Range	Value
Address 0	8-Bit	0-255	025
Address 1	8-Bit	0-255	029

EEPROM Memory Map

Read Operation

- ✓ **No Wear:** Reading does not degrade memory.

Charge Pump

- ⚡ **High Voltage** for Erase & Write

ATmega328P

Flash Memory Program Code

SRAM Temporary Data

EEPROM Persistent Storage

Write Operation

- ✓ **Erase:** Set bits to 1
- ✓ **Write:** New Value
- ⚠ **Wear:** Write Cycle Causes Wear

Important Timing

- 🕒 **EEPROM.write():** 3.3 ms Delay
- 🛑 **Interrupts Disabled** During Write

Managing EEPROM Writes

- ✓ **Minimize Writes** to Extend Lifespan
- ✓ **Typical Limit:** 100,000 Write Cycles

Best Practice: Use `setup()` for Bulk Writes

EEPROM Size: 1024 Bytes (Addresses 0~1023)

Address 0	8-Bit	0-255	025																
Address 1	8-Bit	0-255	029																

EEPROM Memory Map

- Read Operation —

-  **No Wear:** Reading does not degrade memory.

Charge Pump

-  **High Voltage for Erase & Write**

ATmega328P

Write Operation

- ✓ **Erase:** Set bits to 1
- ✓ **Write:** New Value
- ▶ **Wear:** Write Cycle Causes Wear

Important Timing

- EEPROM.write(): 3.3 ms Delay**
-
- 1 Interrupts Disabled** During Write

– Managing EEPROM Writes

- ✓ **Minimize Writes to Extend Lifespan**
- ✓ **Typical Limit: 100,000 Write Cycles**

Best Practice: Use `setup()` for Bulk Writes

Flash Memory Program Code

SRAM Temporary Data

EEPROM Persistent Storage

The Critical Difference – *write()* vs *update()*

Read Operation



No wear; retrieves data without modifying the memory.

Write Operation



Writes data even if the value is the same; causes wear over time.

Update Operation



Writes only if the new value, differs from the current value, minimizing wear.

Comparison of *write()* and *update()*



Write: Writes every time, causing wear.



Update: Only writes when necessary, reducing wear.

Best Practice: Always prefer **update()** unless you have a specific reason to use **write()**.

Playing with Memory

Write `write()`

```
#include <EEPROM.h>

void setup() {
  Serial.begin(9600);
}

void loop() {
  // This will wear out EEPROM quickly
  EEPROM.write(0, 42);
  delay(100);
}
```

Update `update()`

```
#include <EEPROM.h>

void setup() {
  Serial.begin(9600);
}

void loop() {
  /* This writes only once,
  then skips subsequent writes */
  EEPROM.update(0, 42);
  delay(100);
}
```

HW : Black Box Recorder (Boot Counter)

Create a boot counter that increments every time the Arduino powers on, and store it in EEPROM. This demonstrates non-volatile storage.

EEPROM Lifetime Management & Strategies

Endurance Calculation



- Each byte has 100,000 write cycles.
- Writing to different addresses does not affect each other.
- If you write to the same address every second, it lasts ~27.7 hours.
- *With `update()`*, if the value changes rarely, the lifespan extends dramatically.

Wear-Leveling



For data that changes frequently (e.g. counter, log), you can implement wear-leveling:

- Use multiple addresses in a circular buffer.
- Store a "current index" in another EEPROM cell.
- Each time you *write*, advance the index.

System Reliability – The Watchdog Timer (Deep Dive)

The Watchdog Timer (WDT) is a dedicated hardware timer inside the ATmega328P. Its sole purpose is to monitor software execution and reset the microcontroller if the software fails to “pet” it.

Key Features:



Independent 128 kHz Oscillator



Reset or Interrupt Options



Timeouts: 15 ms to 8s



Can Only Be Disabled by Reset

How It Works:



Enable WDT with Timeout Period



Counter Runs Continuously



Periodic `wdt_reset()` Call



Timeout Reached? Trigger Reset!



Microcontroller Reboots

**Hardware-Level Failsafe for Recovering
from Software Hangs & Deadlocks**

[illegible]

On the ATmega328P, the WDT is controlled by the Watchdog Timer Control Register (WDTCR).

Bit	Name	Description
7	WDIF	Watchdog Interrupt Flag (set when timeout occurs in interrupt mode)
6	WDIE	Watchdog Interrupt Enable (if set, interrupt instead of reset)
5	WDP3	Timeout Prescaler Bit 3
4	WDCE	Watchdog Change Enable (must be set to change prescaler)
3	WDE	Watchdog System Reset Enable (if set, reset on timeout)
2	WDP2	Timeout Prescaler Bit 2
1	WDP1	Timeout Prescaler Bit 1
0	WDPO	Timeout Prescaler Bit 0

WDTCR Register

[illegible]

Lab 2 : Intentional Crash & Rescue

Demonstrate the Watchdog Timer by deliberately causing a software hang and letting the WDT automatically reset the system.

HW : Combining EEPROM & WDT – Self-Healing with Persistent Crash Log

Use EEPROM to count how many times the watchdog has reset. If the count exceeds a threshold, disable the watchdog to prevent an endless reboot loop.